HERENCIA

Pilares de la Programación Orientada a Objetos

Desarrollo de Sistemas Orientado a Objetos · 2026

Prof. Esteban Santini · 2026

¿QUÉ ES LA HERENCIA?

El mecanismo de reutilización más poderoso de la POO

La herencia es un mecanismo que permite que una clase (clase hija o subclase) adquiera los atributos y métodos de otra clase (clase padre o superclase). Modela relaciones del tipo “ES UN”: un Auto ES UN Vehículo, un Perro ES UN Animal.

La subclase hereda todo el comportamiento de la superclase y puede extenderlo o modificarlo. En Java, la relación de herencia se establece con la palabra clave **extends.**

BENEFICIOS DE LA HERENCIA

¿Por qué aplicar herencia en nuestros diseños?

Reutilización de código: los atributos y métodos definidos en la clase padre no necesitan reescribirse en la clase hija, eliminando la duplicación.

Extensibilidad: las subclases pueden agregar nuevos atributos y métodos sin modificar la clase padre. Jerarquías claras: el modelo refleja relaciones reales del dominio. Mantenibilidad: los cambios en el padre se propagan a todas las subclases.

HERENCIA ENTRE CLASES CONCRETAS

La base: una clase extiende directamente a otra

Una clase concreta es aquella que puede instanciarse directamente. La herencia entre clases concretas es el caso más simple: la subclase hereda todos los atributos y métodos public y protected de la superclase, y puede agregar los propios.

En Java: public class Auto extends Vehiculo { ... }. La clase hija incorpora nuevos atributos y métodos propios, mientras conserva todo lo heredado. Un Auto puede arrancar() porque lo hereda de Vehiculo.

HERENCIA CONCRETA EN JAVA

Vehiculo → Auto

```
public class Vehiculo {
    protected String marca;
    protected int anio;
    public Vehiculo(String m, int a) { marca=m; anio=a; }
    public void arrancar() {
        System.out.println("Arrancando: " + marca);
    }
}

public class Auto extends Vehiculo {
    private int puertas;
    public Auto(String m, int a, int p) {
        super(m, a);
        this.puertas = p;
    }
    public int getPuertas() { return puertas; }
}
```

CLASES ABSTRACTAS

Cuando la clase padre no puede instanciarse por sí sola

Una clase abstracta NO puede instanciarse directamente. Se declara con abstract y sirve para definir una estructura común que las subclases deben completar. Una clase abstracta puede tener métodos concretos (con cuerpo, heredados) y métodos abstractos (sin cuerpo, que cada subclase DEBE implementar).

En Java: public abstract class Figura { public abstract double calcularArea(); }. Si una subclase no implementa todos los métodos abstractos, también debe declararse abstract. Las clases abstractas modelan conceptos genéricos que no tienen sentido como objetos concretos.

CLASES ABSTRACTAS EN JAVA



Figura (abstract) → Circulo

```java
public abstract class Figura {
    protected String color;
    public Figura(String color) { this.color = color; }
    public abstract double calcularArea();
    public void mostrarInfo() {
        System.out.println("Area: " + calcularArea());
    }
}

public class Circulo extends Figura {
    private double radio;
    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }
    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

MODIFICADORES DE VISIBILIDAD EN HERENCIA

private

Solo accesible dentro de la clase que lo define. La subclase NO puede acceder directamente; debe usar getters y setters. Úsalo para datos sensibles del estado interno.

public

Accesible desde cualquier clase. Los métodos que forman parte de la interfaz pública deben ser public. Las subclases los heredan y pueden sobreescribirlos.

protected

Accesible dentro de la clase y en todas sus subclases. Es el modificador clave en herencia: permite que la subclase acceda directamente al atributo sin romper la encapsulación.

Regla de Oro

Atributos: declarar como private o protected. Métodos: declarar como public. Usar protected cuando la subclase necesita acceso directo al estado del padre.

LA REFERENCIA super

Acceder al comportamiento de la clase padre desde la subclase

La palabra clave super permite referenciar la clase padre desde la subclase. Tiene dos usos: 1) En el constructor — super(args) llama al constructor del padre y debe ser la primera instrucción del constructor de la subclase. Permite inicializar correctamente los atributos heredados.

2) En métodos — super.metodo() invoca la versión del método definida en el padre. Es fundamental en la sobreescritura parcial: reutiliza el comportamiento base y lo extiende. Ejemplo: super.presentarse() + “, Gerente de ” + area.

TRATAMIENTO DE MÉTODOS EN HERENCIA

CON SOBREESCRITURA

- **Sobreescritura Total**

Se reescribe el cuerpo del método desde cero, ignorando completamente el comportamiento del padre. Se usa @Override y se define un nuevo cuerpo completo.

- **Sobreescritura Parcial**

Se usa super.metodo() para reutilizar el comportamiento del padre y se agrega lógica propia. Combina herencia con extensión.

- **¿Cuándo usarla?**

Cuando la subclase necesita comportamiento diferente al del padre (total) o cuando necesita extenderlo (parcial).

SIN SOBREESCRITURA

- **Herencia Directa**

El método del padre se hereda tal cual, sin redefinición en la subclase. La subclase lo usa directamente.

- **Comportamiento compartido**

Todas las subclases tienen el mismo comportamiento heredado del padre, que no necesita cambios.

- **¿Cuándo usarla?**

Cuando el comportamiento del padre es completamente correcto para la subclase y no requiere cambios.

SOBREESCRITURA TOTAL

@Override — cuerpo completamente nuevo

```
public class Animal {
    public void hacerSonido() {
        System.out.println("...");
    }
}

public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        // Nuevo cuerpo: ignora completamente al padre
        System.out.println("Guau!");
    }
}

// Uso:
Animal a = new Perro();
a.hacerSonido();   // Imprime: Guau!
```

SOBREESCRITURA PARCIAL

@Override + super.metodo() — reutilizar y extender

```
public class Empleado {
    protected String nombre;
    public String presentarse() {
        return "Soy " + nombre;
    }
}

public class Gerente extends Empleado {
    private String area;
    @Override
    public String presentarse() {
        // Reutiliza comportamiento del padre...
        String base = super.presentarse();
        // ...y agrega logica propia
        return base + ", Gerente de " + area;
    }
}
```

SIN SOBREESCRITURA

El método se hereda tal cual del padre

```
public class Vehiculo {
    protected String marca;
    public void arrancar() {
        System.out.println(marca + " arrancando...");
    }
}

public class Auto extends Vehiculo {
    private int puertas;
    // arrancar() NO se redefine en Auto.
    // Se hereda directamente de Vehiculo.
    // El comportamiento del padre es suficiente.
}

// Uso:
Auto a = new Auto();
a.arrancar();    // Usa el metodo heredado
```



LA HERENCIA ES REUTILIZAR Y EXTENDER

“Una subclase no repite lo que ya existe. Lo hereda, lo adapta y lo supera.”
La Herencia nos permite construir jerarquías coherentes, reducir la duplicación de código y diseñar sistemas extensibles y mantenibles.